

Prompt Protokoll

Copy and paste as many blocks as you need. Track every single AI Interaction

AI Model Chat GPT	Date 30.09.2026
Prompt: Erstelle mir aus folgender Beschreibung einen Algorithmus in der Sprache Java [Algorithmus soll überprüfen wie effektiv eingesetzte Attacken sind und dementsprechend Schaden zufügen. Anfang: Definiert wir der Spieler und der Gegner haben jeweils ein Tier von einem bestimmten Typen. System weiß welche attacke unser Tier und der Gegner wählen und einsetzt. Wenn eine attacke passiert schaut sich der Algorithmus die attacke an und entscheidet. Wenn die Attacke von mindestens einem Tier ein "Konter" ist, endet die Runde. Es geht in die nächste Runde. Wenn die Attacken beide Schaden verursachen, wertet der Algorithmus für beide Tiere sowohl des Spielers als auch des Gegners den Schaden gleichzeitig aus. Schaden wird wie folgt berechnet: Schritt 1 - Es wird verglichen, ob der Typ des Tiers und der Attacke gleich ist. Fall 1 = Typ des Tiers ist gleich wie Typ der Attacke: +15% auf die Attribute der jeweiligen Attacke (defense und attack werte) Fall 2 = Typ des Tiers ist nicht gleich Typ der Attacke: die Grundwerte der Attacke (defense und attack Werte) kommen unverändert zur Geltung. Es springt zu Schritt 3. Schritt 2 – die Effektivität wird überprüft, dabei werden die Typen der beiden Tiere vergleichen. Fall 1 = Typ von Tier ist effektiv gegen Gegner: Tier bekommt +20% auf Attacke (defense und attack Werte) Fall 2 = Typ von Tier ist neutral gegenüber Gegner: Werte kommen unverändert zur Geltung. Schritt 3: Schaden wird als zahl (integer) output ausgegeben. Schritt 4: Health bar – Schaden = neuer Health bar. Schritt 5: Health bar ist kleiner gleich 0, dann Spiel zu ende, sonst wieder von vorne (Schritt 1).]. Die Eingabe ist eine Liste mit folgenden Daten, ich will, dass ... Als Ergebnis will ich eine neue Liste ... Analyse: Wir haben vergessen zu definieren wie der Defense wert funktioniert, darauf hat uns Chat auch hingewiesen. Zusätzlich wollte er das wir Attacken runden am Anfang eingeben, wir nehmen an, dass das für den Test okay ist. Wir definieren unseren Prompt neu.	

Prompt

Erstelle mir aus folgender Beschreibung einen Algorithmus in der Sprache Java

[Anfang: Definieren wir der Spieler und der Gegner haben jeweils ein Tier von einem bestimmten Typen. Beide Tiere starten mit 100 Punkten im Health bar. Das System weiß welche attacke unser Tier und der Gegner wählen und einsetzt.

Wenn eine attacke passiert schaut sich der Algorithmus die attacke an und entscheidet. Wenn die Attacke von mindestens einem Tier ein "Konter" ist, endet die Runde. Es geht in die nächste Runde.

Wenn die Attacken beide Schaden verursachen, wertet der Algorithmus für beide Tiere sowohl des Spielers als auch des Gegners den Schaden gleichzeitig aus.

Schaden wird wie folgt berechnet:

Schritt 1 - Es wird verglichen, ob der Typ des Tiers und der Attacke gleich ist.

Fall 1 = Typ des Tiers ist gleich wie Typ der Attacke: +15% auf die Attribute der jeweiligen Attacke (defense und attack werte)

Fall 2 = Typ des Tiers ist nicht gleich Typ der Attacke: die Grundwerte der Attacke (defense und attack Werte) kommen unverändert zur Geltung. Es springt zu Schritt 3.

Schritt 2 – die Effektivität wird überprüft, dabei werden die Typen der beiden Tiere verglichen.

Fall 1 = Typ von Tier ist effektiv gegen Gegner: Tier bekommt +20% auf Attacke (defense und attack Werte)

Fall 2 = Typ von Tier ist neutral gegenüber Gegner: Werte kommen unverändert zur Geltung.

Schritt 3: Schaden wird als zahl (integer) output ausgegeben. Und der Gegner wird berechnet. Wenn beide Tiere bis zu diesem Schritt berechnet sind geht es zu Schritt 4 weiter.

Schritt 4: Für beide Tiere wird das folgende berechnet: Schaden von Tier minus Defense von Gegner = tatsächlicher Schaden.

Schritt 5: Für beide Tiere wird das folgende berechnet: Health bar Gegner – Schaden = neuer Health bar von Gegner.

Schritt 6: Für beide Tiere wird abgeglichen: Health bar ist kleiner gleich 0, dann Spiel zu ende, sonst wieder von vorne (Schritt 1).]. Die Eingabe ist eine Liste mit folgenden Daten, Typ von Tier mit 4 verschiedenen Attacken die jeweils eine bestimmte punkte anzahl für defense und attack haben.

Als Ergebnis will ich eine neue Liste.

Algorythmus:

Ja. Ich würde den Algorithmus so aufbauen, dass du eine **Liste mit Tieren** übergibst und als Ergebnis eine **neue Liste mit den berechneten Kampfergebnissen** erhältst.

Ich nehme dabei folgende Struktur an:

- Jedes Tier hat:
 - Typ
 - Health = 100
 - 4 Attacken
- Jede Attacke hat:
 - Name
 - Typ
 - Attack-Wert
 - Defense-Wert
 - konter = true/false
- Zusätzlich müssen wir wissen, welche Typen gegeneinander effektiv sind.
- Prozentwerte werden am Ende auf int gerundet.

Hier ist eine mögliche Java-Lösung, bewusst relativ einfach gehalten:

```
import java.util.ArrayList;
import java.util.List;

public class KampfAlgorithmus {

    // ----- TIER -----

    static class Tier {
        String name;
        String typ;
        int health;
        List<Attacke> attacken;

        Tier(String name, String typ, List<Attacke> attacken) {
            this.name = name;
            this.typ = typ;
            this.health = 100;
            this.attacken = attacken;
        }
    }

    // ----- ATTACKE -----

    static class Attacke {
        String name;
        String typ;
        int attack;
        int defense;
        boolean konter;

        Attacke(String name,
                String typ,
```

```

        int attack,
        int defense,
        boolean konter) {

        this.name = name;
        this.typ = typ;
        this.attack = attack;
        this.defense = defense;
        this.konter = konter;
    }
}

// ----- ERGEBNIS -----

static class KampfErgebnis {
    String tier;
    int schaden;
    int neueHealth;

    KampfErgebnis(String tier, int schaden, int neueHealth) {
        this.tier = tier;
        this.schaden = schaden;
        this.neueHealth = neueHealth;
    }

    @Override
    public String toString() {
        return tier
            + " | Schaden: " + schaden
            + " | Health: " + neueHealth;
    }
}

// =====
// HAUPTALGORITHMUS
// =====

static List<KampfErgebnis> kampf(
    Tier spieler,
    Tier gegner,
    Attacke spielerAttacke,
    Attacke gegnerAttacke) {

    List<KampfErgebnis> ergebnisListe = new ArrayList<>();

    // -----
    // KONTER
    // -----

    if (spielerAttacke.konter || gegnerAttacke.konter) {

        System.out.println("Konter!");
    }
}

```

```

        System.out.println("Runde wird beendet.");

        return ergebnisListe;
    }

    // -----
    // SCHRITT 1 + 2
    // Angriffswerte berechnen
    // -----

    int spielerAttack =
        berechneAttackWert(
            spieler,
            gegner,
            spielerAttacke
        );

    int spielerDefense =
        berechneDefenseWert(
            spieler,
            gegner,
            spielerAttacke
        );

    int gegnerAttack =
        berechneAttackWert(
            gegner,
            spieler,
            gegnerAttacke
        );

    int gegnerDefense =
        berechneDefenseWert(
            gegner,
            spieler,
            gegnerAttacke
        );

    // -----
    // SCHRITT 3
    // Berechnete Werte ausgeben
    // -----

    System.out.println(
        spieler.name
        + " Attack: "
        + spielerAttack
    );

    System.out.println(
        gegner.name

```

```

        + " Attack: "
        + gegnerAttack
    );

    // -----
    // SCHRITT 4
    // tatsächlichen Schaden berechnen
    // -----

    int schadenAnGegner =
        spielerAttack - gegnerDefense;

    int schadenAnSpieler =
        gegnerAttack - spielerDefense;

    // Schaden darf nicht negativ sein
    if (schadenAnGegner < 0) {
        schadenAnGegner = 0;
    }

    if (schadenAnSpieler < 0) {
        schadenAnSpieler = 0;
    }

    // -----
    // SCHRITT 5
    // Health reduzieren
    // Beide Schäden werden erst berechnet und DANACH
    // angewendet -> dadurch gleichzeitig
    // -----

    gegner.health =
        gegner.health - schadenAnGegner;

    spieler.health =
        spieler.health - schadenAnSpieler;

    // Health soll nicht unter 0 angezeigt werden
    if (gegner.health < 0) {
        gegner.health = 0;
    }

    if (spieler.health < 0) {
        spieler.health = 0;
    }

    // -----
    // NEUE LISTE ERSTELLEN
    // -----

```

```

        ergebnisListe.add(
            new KampfErgebnis(
                spieler.name,
                schadenAnSpieler,
                spieler.health
            )
        );

        ergebnisListe.add(
            new KampfErgebnis(
                gegner.name,
                schadenAnGegner,
                gegner.health
            )
        );

// -----
// SCHRITT 6
// Prüfen ob Kampf beendet
// -----

if (spieler.health <= 0 && gegner.health <= 0) {

    System.out.println("Unentschieden!");

} else if (spieler.health <= 0) {

    System.out.println(
        gegner.name + " gewinnt!"
    );

} else if (gegner.health <= 0) {

    System.out.println(
        spieler.name + " gewinnt!"
    );

} else {

    System.out.println(
        "Beide Tiere leben noch."
    );

}

return ergebnisListe;
}

// =====
// ATTACK-WERT BERECHNEN
// =====

```

```

static int berechneAttackeWert(
    Tier tier,
    Tier gegner,
    Attacke attacke) {

    double wert = attacke.attack;

    // -----
    // SCHRITT 1
    // Tier-Typ == Attacken-Typ
    // -> +15 %
    // -----

    if (tier.typ.equals(attacke.typ)) {

        wert = wert * 1.15;
    }

    // -----
    // SCHRITT 2
    // Typ effektiv gegen Gegner
    // -> +20 %
    // -----

    if (istEffektiv(tier.typ, gegner.typ)) {

        wert = wert * 1.20;
    }

    // Integer Ergebnis
    return (int) Math.round(wert);
}

// =====
// DEFENSE-WERT BERECHNEN
// =====

static int berechneDefenseWert(
    Tier tier,
    Tier gegner,
    Attacke attacke) {

    double wert = attacke.defense;

    // gleicher Typ
    // -> +15 %
    if (tier.typ.equals(attacke.typ)) {

```

```

        wert = wert * 1.15;
    }

    // effektiv gegen Gegner
    // -> +20 %
    if (istEffektiv(tier.typ, gegner.typ)) {

        wert = wert * 1.20;
    }

    return (int) Math.round(wert);
}

// =====
// TYP-EFFEKTIVITÄT
// =====

static boolean istEffektiv(
    String angreiferTyp,
    String gegnerTyp) {

    /*
    Beispiel:

    Feuer > Pflanze
    Pflanze > Wasser
    Wasser > Feuer

    Diese Liste könnt ihr natürlich
    durch eure echten Tier-Typen ersetzen.
    */

    if (angreiferTyp.equals("Feuer")
        && gegnerTyp.equals("Pflanze")) {

        return true;
    }

    if (angreiferTyp.equals("Pflanze")
        && gegnerTyp.equals("Wasser")) {

        return true;
    }

    if (angreiferTyp.equals("Wasser")
        && gegnerTyp.equals("Feuer")) {

        return true;
    }
}

```

```
        return false;
    }
}
```

Beispiel für eure Eingabeliste

Ein Feuer-Tier könnte zum Beispiel so erstellt werden:

```
List<KampfAlgorithmus.Attacke> attackenFeuer =
    new ArrayList<>();
```

```
attackenFeuer.add(
    new KampfAlgorithmus.Attacke(
        "Feuerball",
        "Feuer",
        30,
        10,
        false
    )
);
```

```
attackenFeuer.add(
    new KampfAlgorithmus.Attacke(
        "Flammenwand",
        "Feuer",
        15,
        25,
        false
    )
);
```

```
attackenFeuer.add(
    new KampfAlgorithmus.Attacke(
        "Biss",
        "Normal",
        20,
        10,
        false
    )
);
```

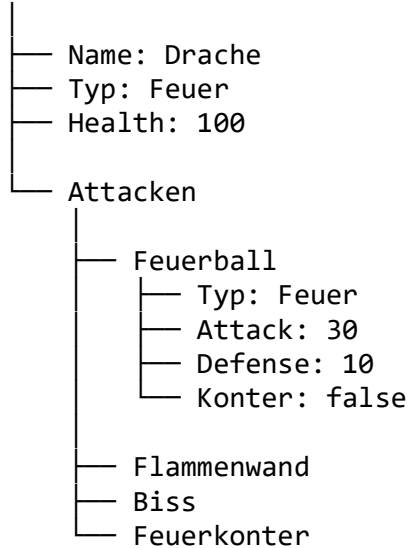
```
attackenFeuer.add(
    new KampfAlgorithmus.Attacke(
        "Feuerkonter",
        "Feuer",
        0,
        30,
        true
    )
);
```

```
KampfAlgorithmus.Tier spieler =
    new KampfAlgorithmus.Tier(
```

```
"Drache",  
"Feuer",  
    attackenFeuer  
);
```

Die Eingabedaten wären also gedanklich:

Tier



Was der Algorithmus genau macht

Bei einer Attacke mit beispielsweise:

Tier-Typ: Feuer

Attacken-Typ: Feuer

Attack: 30

Defense: 10

kommt zuerst der **15%-Bonus**, weil Tier und Attacke denselben Typ haben:

Attack:

$$30 \times 1,15 = 34,5$$

Danach wird überprüft, ob beispielsweise:

Feuer effektiv gegen Pflanze

ist.

Dann kommen noch einmal **20 %** dazu:

$$34,5 \times 1,20 = 41,4$$

→ gerundet = 41

Das Gleiche passiert mit Defense.

Angenommen der Gegner hat danach:

Defense = 15

dann ist der tatsächliche Schaden:

$41 - 15 = 26$

Der Gegner hatte:

100 Health

also:

$100 - 26 = 74$ Health

Die neue Ergebnisliste könnte dann etwa so aussehen:

```
[
    SpielerTier | Schaden: 18 | Health: 82,
    GegnerTier  | Schaden: 26 | Health: 74
]
```

Ein wichtiger Punkt aus deiner Beschreibung ist hier ebenfalls umgesetzt: **Die Angriffe werden gleichzeitig ausgewertet.** Der Algorithmus berechnet deshalb zuerst

```
int schadenAnGegner = ...
int schadenAnSpieler = ...
```

und verändert **erst danach** beide Health-Werte. Dadurch kann beispielsweise auch ein Fall entstehen, bei dem beide Tiere in derselben Runde auf 0 Health fallen.

Analyse:

So wie Chat das Ergebnis erklärt klingt es richtig. Ich lasse ihn das mir nochmal als "praxis erklären" siehe nächsten prompt.

AI Model	Date
Prompt Chat spiel mit mit diesem code und zwei erfundenen tieren einmal durch und beschreibe wie jeder Schritt die werte verändern würde Analyse: Das ist jetzt genau was ich wollte, also hat der Prompt von zuvor gepasst.	